Supply Chain Network Credit Contagion

A Comprehensive Technical Report

Modeling Systemic Credit Risk Propagation
in Automotive Supply Chain Networks
Using Graph Neural Networks

Author: Harshvardha Upadhyaya

July 2026

Technologies: Python · PyTorch Geometric · NetworkX · FastAPI · React

Domain: Financial Risk Management · Graph Neural Networks · Supply Chain Analytics

Contents

1 Introduction — What Is This Project About? 3
- 1.1 The Problem: Why Companies Don't Fail Alone . . . 3
- 1.2 What We Built . . . 3
- 1.3 Key Concepts for Beginners . . . 4
  - 1.3.1 What Is a Graph? . . . 4
  - 1.3.2 What Is a Neural Network? . . . 4
  - 1.3.3 What Is a Graph Neural Network (GNN)? . . . 4
  - 1.3.4 What Is the Altman Z-Score and Its Financial Terms? . . . 5

2 Technology Stack — Tools We Used and Why 8
- 2.1 Overview . . . 8
- 2.2 How These Tools Connect . . . 9

3 Project Folder Structure 10
- 3.1 Directory Layout . . . 10

4 Phase 1 — Setup and Planning 12
- 4.1 Goal . . . 12
- 4.2 Key Decisions Made . . . 12
- 4.3 Edge Type Design . . . 12

5 Phase 2 — Data Collection and Engineering 13
- 5.1 Raw Data Sources . . . 13
  - 5.1.1 Supply Chain Data — Who Supplies Whom . . . 13
  - 5.1.2 Financial Data — Company Health Metrics . . . 13
- 5.2 Step 11: Entity Resolution . . . 14
  - 5.2.1 The Problem . . . 14
  - 5.2.2 The Solution: Three-Strategy Pipeline . . . 14
- 5.3 Step 12: Data Validation . . . 16
- 5.4 Step 13: Altman Z-Score Computation . . . 16
  - 5.4.1 The Proxy Challenge . . . 16
  - 5.4.2 Results in Our Dataset . . . 17

6 Phase 3 — Graph Construction and Feature Engineering 18
- 6.1 Goal . . . 18
- 6.2 Step 15: Building the Graph . . . 18
- 6.3 Step 16: Computing Topological Metrics . . . 19
- 6.4 Step 17: Merging Features into Node Matrix X . . . 19
- 6.5 Step 18: Computing Edge Weights W . . . 20

7 Phase 4 — Machine Learning Model **21**
7.1 Goal . . . 21
7.2 Step 19: Baseline Model (Logistic Regression) . . . 21
7.2.1 Baseline Results . . . 22
7.3 Step 20: PyTorch Geometric Dataset . . . 22
7.3.1 Why We Need This Conversion . . . 22
7.3.2 Key Design Decisions in the Code . . . 22
7.3.3 Train/Test Split and Scaling . . . 23
7.4 Step 21: GNN Architecture . . . 23
7.4.1 Why 2 Layers? . . . 25
7.4.2 Why Dropout = 0.5? . . . 25
7.5 Step 22: Training Loop . . . 25

8 Phase 5 — Contagion Simulation and Stress Testing **27**
8.1 Goal . . . 27
8.2 The Core Idea . . . 27
8.3 Shock Injection Engine . . . 28
8.3.1 Node Removal (SCR-NR) . . . 28
8.3.2 Edge Removal (SCR-ER) . . . 28
8.4 Risk Metrics Calculation . . . 28

9 Phase 6 — API and Dashboard **30**
9.1 Goal . . . 30
9.2 FastAPI Backend . . . 30
9.2.1 Performance Optimization: Global State Cache . . . 30
9.2.2 CORS Configuration . . . 31
9.3 React Frontend . . . 31
9.3.1 Graph Visualization . . . 31

10 Results and Discussion **33**
10.1 Dataset Statistics . . . 33
10.2 Model Comparison . . . 33
10.3 Key Insight: Why Graph Structure Matters . . . 34

11 Limitations and Future Work **35**
11.1 Current Limitations . . . 35
11.2 Planned Enhancements (Phases 7–10) . . . 35

A Glossary of Acronyms **36**

Chapter 1

Financial Context, Theory, and Project Rationale

This chapter provides an exhaustive, first-principles treatment of every financial concept, risk-management framework, and economic rationale that underpins this project. It is designed so that a reader with a finance background can assess the depth of understanding behind each design decision, and so that the project author can confidently discuss the financial theory during an interview.

1.1 The Foundational Problem: Credit Risk in Interconnected Economies

1.1.1 What Is Credit Risk?

Credit risk is the probability that a borrower or counterparty will fail to meet its financial obligations — repaying a loan, honouring a bond coupon, or settling a trade payable. It is the single largest source of risk for commercial banks and institutional lenders worldwide.

Credit risk manifests in several forms:

- **Default risk:** The outright failure of a counterparty to pay (Chapter 7 bankruptcy, liquidation).
- **Migration risk:** A deterioration in creditworthiness (e.g., a downgrade from BBB to BB) that does not constitute default but increases the probability of future default and reduces the market value of outstanding obligations.
- **Concentration risk:** Excessive exposure to a single counterparty, sector, or geography, so that one adverse event triggers disproportionate portfolio losses.
- **Recovery risk:** Uncertainty about how much a creditor will recover after a default occurs (the inverse of Loss Given Default).
- **Country / sovereign risk:** The risk that a country's economic or political environment impairs private-sector repayment capacity (capital controls, currency devaluation, sanctions).

1.1.2 Why Traditional Credit Risk Models Fail for Supply Chains

The dominant paradigm in credit risk management — used by virtually every major bank, rating agency, and credit insurer — treats each obligor as an **isolated, independent entity**. The three most widely deployed frameworks illustrate this:

1. **Moody's KMV (Expected Default Frequency):** Models each firm as a call option on its own assets using the Merton structural model. The firm's distance-to-default is computed from its asset value, asset volatility, and debt maturity — all endogenous to the single firm. Correlations between firms are captured only through shared exposure to macroeconomic factors (GDP growth, interest rates), not through direct business linkages.
2. **CreditMetrics (J.P. Morgan):** Estimates portfolio credit risk by simulating correlated rating transitions. Firm-to-firm correlation is derived from equity return correlations, which are themselves driven by shared sector and geographic factors — again, not by actual supply chain dependencies.
3. **CreditRisk+ (Credit Suisse):** An actuarial model that treats defaults as Poisson-distributed events driven by a small number of systematic risk factors. It is computationally efficient but structurally incapable of capturing cascading, network-mediated contagion.

The critical gap: None of these models incorporate the *topology* of real-world business relationships. They cannot answer questions like:

- "If TSMC (the world's largest semiconductor foundry) shuts down, what happens to the 50+ automotive OEMs that depend on its chips?"
- "If a tier-2 battery materials supplier defaults, which EV manufacturers are exposed through their tier-1 battery cell suppliers?"
- "Which company in the network, if removed, would cause the maximum cascading increase in expected losses across the portfolio?"

This project exists to fill precisely this gap.

1.1.3 Empirical Evidence: Why Interconnectedness Matters

The claim that supply chain interconnectedness amplifies credit risk is not theoretical speculation — it is supported by extensive empirical evidence:

Event	Year	Observed Contagion Effect
Tōhoku Earthquake	2011	Toyota lost 800,000 units of production. Renesas Electronics (semiconductor supplier) was offline for 3 months, affecting 60% of global automotive MCU supply. Downstream OEMs across 4 continents experienced production halts.
Thai Floods	2011	Destroyed hard-disk-drive supply chains (Western Digital, Seagate). HDD prices rose 80% globally. Companies 3–4 tiers removed experienced inventory write-downs.
COVID-19 Shutdowns	2020	Wuhan supplier shutdowns cascaded within 4 weeks to European auto plants. Ford, GM, and VW idled factories due to missing <$5 components.
Global Chip Shortage	2021–23	TSMC, Samsung, and GlobalFoundries capacity constraints caused $210 billion in lost automotive revenue globally. Companies with diversified sourcing (Toyota) recovered faster than those with concentrated suppliers (GM).
Red Sea Disruption	2024	Houthi attacks on shipping forced 90% of container traffic to reroute around the Cape of Good Hope, adding 10–14 days to Asia–Europe transit. Just-in-time supply chains experienced cascading delays.

Table 1.1: Historical evidence of supply chain credit contagion events.

Prior academic microdata studies have quantified the amplification:

- **Expected Loss (EL)** amplified by $\sim$5.2$\times$ when network effects are included vs. isolated analysis.
- **Value at Risk (VaR)** amplified by $\sim$6.7$\times$.
- **Expected Shortfall (ES)** amplified by $\sim$4.4$\times$.

These are not edge cases — they represent the *normal operating reality* of modern, globally integrated manufacturing.

1.2 Credit Risk Measurement: The Quantitative Framework

This section defines every financial metric and formula used in this project, from first principles.

1.2.1 Probability of Default (PD)

The **Probability of Default** is the likelihood that an obligor will fail to honour its debt obligations within a specified time horizon (typically 1 year for regulatory purposes).

In this project, PD is determined via two complementary approaches:

1. **Static PD (Altman Z-Score):** A discriminant-analysis-based bankruptcy prediction model that classifies firms into Safe ($Z > 2.99$), Grey ($1.81 < Z \leq 2.99$), or Distressed ($Z \leq 1.81$) zones. We use the distress threshold as a binary label: default_label $= \mathbb{1}[Z \leq 1.81]$. This serves as the *ground truth* for GNN training.
2. **Dynamic PD (GNN-Predicted):** The trained Graph Neural Network outputs a continuous probability $PD_i \in [0, 1]$ for each company i, incorporating both intrinsic financial features and network-mediated contagion effects. This is the *network-adjusted* PD that represents the project's core value proposition.

1.2.2 Loss Given Default (LGD)

LGD represents the fraction of exposure that is permanently lost when a default occurs. It equals $1 -$ Recovery Rate.

- **Our assumption:** $LGD = 0.45$ (45% loss, 55% recovery).
- **Justification:** This is the Basel II/III Foundation IRB prescribed LGD for senior unsecured corporate exposures. Empirical studies by Moody's (1982–2021 data) show average recovery rates of 40–60% for senior unsecured bonds, with a mean of approximately 55%, yielding an LGD of 45%.
- **Why fixed (not modeled):** In a full production system, LGD would itself be modeled as a function of collateral quality, seniority, jurisdiction, and macroeconomic state. For this prototype, using the regulatory standard isolates the effect of the GNN on PD estimation without confounding it with LGD model risk.

1.2.3 Exposure at Default (EAD)

EAD is the total economic exposure to a counterparty at the moment of default.

- **Our formula:** $EAD_i = \text{Revenue}_i \times 0.15$.
- **Justification:** Revenue is a proxy for the economic scale of a firm's operations. The 15% factor approximates the fraction of a firm's annual revenue that is typically outstanding as trade receivables at any point in time (i.e., goods shipped but not yet paid for). For automotive OEMs, trade receivables-to-revenue ratios typically range from 10%–20%, making 15% a reasonable midpoint.
- **Why revenue-based (not balance-sheet):** Detailed accounts-receivable data is not available for all 174 companies (especially private firms like Bosch and ZF). Revenue is universally available and serves as a consistent scale factor.

1.2.4 Expected Loss (EL) — The Core Formula

The **Expected Loss** is the fundamental credit risk metric used by every major bank under Basel regulatory frameworks:

$$\boxed{EL = PD \times LGD \times EAD} \tag{1.1}$$

Interpretation: EL answers: "On average, how many dollars will we lose from this exposure per year?"

Worked example using our data:

- **Ford Motor Company:** Revenue = \$176B, $PD_{\text{baseline}} = 0.32$
- $EAD = 176\text{B} \times 0.15 = \$26.4\text{B}$
- $EL = 0.32 \times 0.45 \times 26.4\text{B} = \$3.80\text{B}$
- After simulating Toyota's default: $PD_{\text{shocked}} = 0.41$
- $EL_{\text{shocked}} = 0.41 \times 0.45 \times 26.4\text{B} = \$4.87\text{B}$
- $\Delta EL = \$1.07\text{B}$ — this is Ford's contribution to Toyota's Systemic Risk Contribution.

1.2.5 Systemic Risk Contribution (SRC)

The **SRC** of a company k quantifies the total additional expected loss imposed on the entire network when company k defaults:

$$\boxed{SRC_k = \sum_{i \neq k} \max\left(0,\ EL'_i - EL_i\right)} \tag{1.2}$$

where EL_i is the baseline expected loss and EL'_i is the expected loss after company k is removed from the network.

Key properties of SRC:

- **Non-negative:** We only count increases in expected loss (the $\max(0, \cdot)$ ensures we ignore companies whose PD coincidentally decreased).
- **Dollar-denominated:** SRC is expressed in absolute currency units (USD), making it directly interpretable by risk managers as "the total additional loss caused by this company's failure."
- **Asymmetric:** $SRC_{\text{Toyota}} \neq SRC_{\text{Denso}}$ even though they are directly connected, because Toyota is a much larger hub with more downstream dependents.
- **Network-sensitive:** A company with mediocre financials but high betweenness centrality (a "chokepoint") can have a higher SRC than a larger company with many alternative supply paths.

1.2.6 Value at Risk (VaR) and Expected Shortfall (ES)

While not yet implemented in this project (planned for Phase 9), these are critical concepts for the finance interviewer:

- **Value at Risk (VaR):** The maximum loss that will not be exceeded with a given confidence level (e.g., 99%) over a specified time horizon. Formally, VaR_α is the α-quantile of the portfolio loss distribution:

$$P(L > VaR_\alpha) = 1 - \alpha \tag{1.3}$$

- **Expected Shortfall (ES):** Also called Conditional VaR (CVaR). It is the average loss in the worst $(1-\alpha)\%$ of scenarios — i.e., "given that we've already exceeded VaR, how bad is it on average?":

$$ES_\alpha = \mathbb{E}[L \mid L > VaR_\alpha] \tag{1.4}$$

- **Why ES is preferred:** VaR tells you the boundary of the "bad zone" but says nothing about how bad things get beyond it. ES is a *coherent risk measure* (it satisfies sub-additivity: diversification always reduces ES, unlike VaR). Basel III/IV has shifted from VaR to ES for the Fundamental Review of the Trading Book (FRTB).

1.3 The Altman Z-Score: History, Theory, and Our Implementation

1.3.1 Historical Context

Edward I. Altman, a professor at NYU Stern School of Business, published the Z-Score model in 1968 in *The Journal of Finance*. He analyzed 66 companies (33 bankrupt, 33 non-bankrupt) from 1946–1965 using **multiple discriminant analysis (MDA)** — a statistical technique that finds the linear combination of variables that best separates two groups.

From an initial set of 22 candidate financial ratios, Altman selected 5 that, in combination, maximized the separation between bankrupt and non-bankrupt firms. The original model achieved:

- 94% classification accuracy on the training sample
- 82% accuracy on a hold-out validation sample
- Type I error (classifying a bankrupt firm as healthy) = 6%
- Type II error (classifying a healthy firm as bankrupt) = 3%

The model has been updated several times:

- **Z'-Score (1983):** Adapted for private (non-public) companies by replacing X_4 (market cap / total liabilities) with book value of equity / total liabilities.
- **Z"-Score (1995):** Further adapted for non-manufacturing and emerging-market firms by dropping X_5 (revenue / total assets) to remove industry bias.
- **Z-Score Plus (2020s):** Altman's commercial model incorporating qualitative factors, macroeconomic adjustments, and sector-specific calibrations.

Our implementation uses the original Z-Score formula (Equation 1.2) with proxy variable derivations (detailed in Chapter 5) because our dataset comprises publicly traded manufacturing companies.

1.3.2 Why the Z-Score Serves as Ground Truth (Not the Prediction Target)

A critical design decision in this project is that the Z-Score is used **only** to create binary training labels (default_label = $\mathbb{1}[Z \leq 1.81]$). The Z-Score itself is **excluded** from the feature matrix X fed to the GNN.

Rationale:

- If the Z-Score were included as an input feature, the GNN would trivially learn the threshold rule $Z \leq 1.81 \rightarrow$ label $= 1$ and achieve near-perfect accuracy — while learning **nothing** about credit contagion.
- By excluding it, the GNN is forced to discover risk patterns from the *component ratios* (revenue, debt, market cap, profitability) and, crucially, from the *network structure* (who supplies whom, how important is this node in the graph).
- This separation of "labeling mechanism" from "learning signal" is analogous to how medical ML systems use biopsy results (ground truth) to label training images, but do not include the biopsy result in the input features.

1.4 Supply Chain Finance: How Credit Risk Propagates Through Networks

1.4.1 The Three Channels of Credit Contagion

Credit contagion in supply chains propagates through three distinct financial channels, each corresponding to an edge type in our graph:

Channel 1: Supply Disruption (Upstream Propagation)

Edge type: `supply` (Supplier $\rightarrow$ Customer)

Mechanism:

1. A supplier S enters financial distress and cannot fulfil orders.
2. The customer C experiences production disruptions: assembly lines idle, delivery commitments missed.
3. C's revenue drops. If C cannot find an alternative supplier quickly, its own working capital deteriorates.
4. C's credit metrics worsen: current ratio falls, debt-to-equity rises, profit margin compresses.
5. If S was C's *sole source* for a critical component (e.g., TSMC for advanced chips), the effect is catastrophic. If C had multiple qualified suppliers, the effect is attenuated.

Real-world example: When Renesas Electronics' Naka factory was damaged in the 2011 Tōhoku earthquake, it supplied 30% of the world's automotive microcontroller units (MCUs). Toyota, Honda, Nissan, and Stellantis all experienced production shutdowns lasting 2–6 months.

What our model captures: The GNN's message-passing mechanism naturally models this: when a supplier node's features change (features zeroed out in a shock simulation), the customer node receives a different "message" in the next GCN layer, shifting its predicted PD upward.

Channel 2: Trade Credit Default (Accounts Receivable Write-Off)

Edge type: `trade_credit` (Creditor → Debtor)

Mechanism:

1. In B2B commerce, suppliers typically ship goods *before* receiving payment. Standard payment terms are Net-30, Net-60, or Net-90 (payment due 30, 60, or 90 days after invoice).
2. At any given time, a supplier has a significant amount of **accounts receivable (AR)** — money owed by customers for goods already delivered.
3. If a customer C defaults, the supplier S must write off the AR as a bad debt expense. This directly reduces S's net income, retained earnings, and working capital.
4. For suppliers with thin profit margins (typical in automotive parts: 3–8% net margin), a large AR write-off can push the supplier itself into distress.
5. The AR write-off reduces S's Z-Score components: X_1 (working capital / total assets) falls, X_2 (retained earnings / total assets) falls, and X_3 (EBIT / total assets) falls.

Scale of exposure: In the automotive sector, trade credit exposure between a tier-1 supplier and a major OEM can range from \$50M to \$2B, representing 5–20% of the supplier's annual revenue.

What our model captures: Trade credit edges carry higher contagion weights (w_{ij}) in our edge weight formula (Equation 6.1), reflecting the direct financial exposure.

Channel 3: Equity Contagion (Ownership Exposure)

Edge type: `ownership` (Parent → Subsidiary)

Mechanism:

1. Parent companies hold equity stakes in subsidiaries and affiliates. For example, Toyota owns 24.7% of Denso and 20.1% of Aisin.
2. If a subsidiary enters distress, the parent must:
   - Write down the equity investment (reducing book value of assets)
   - Potentially inject emergency capital (increasing debt)
   - Absorb reputational damage (reducing market capitalization)
3. Conversely, if a parent enters distress, subsidiaries lose their financial backstop, credit guarantee, and often their largest customer.

What our model captures: Ownership edges receive a bonus term in the edge weight formula, increasing the contagion transmission probability between parent and subsidiary nodes.

1.4.2 The Cascading Amplification Effect

The three channels above interact in a **cascading feedback loop**:

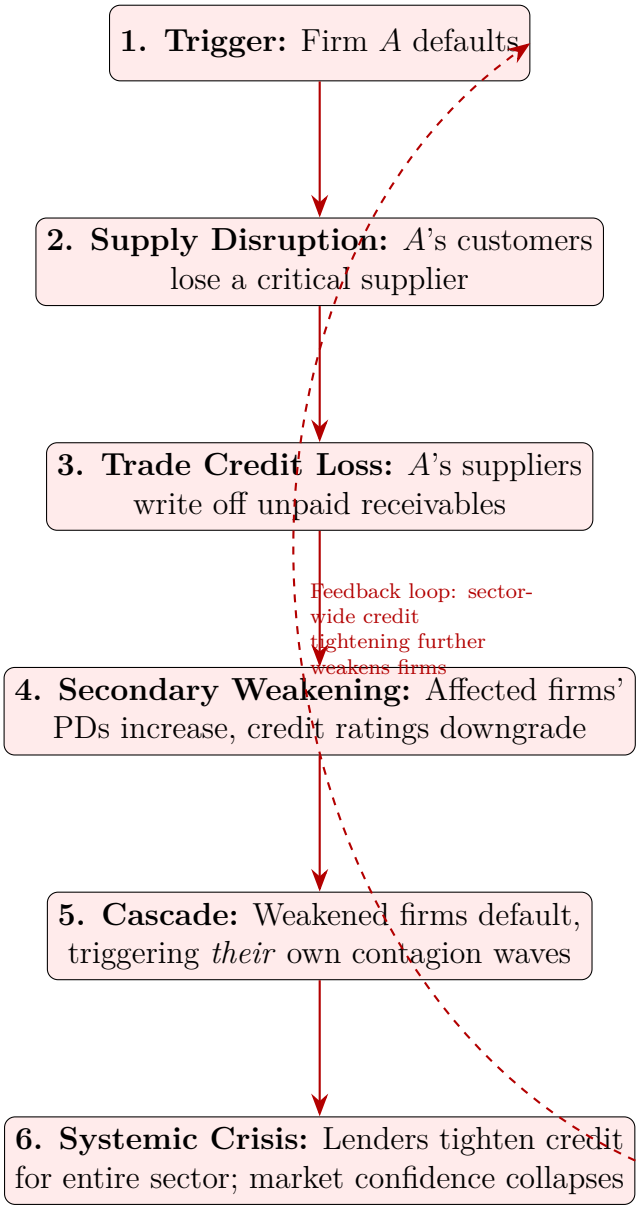


Figure 1.1: The cascading credit contagion feedback loop in supply chain networks.

Key insight for the interviewer: This cascading amplification is why empirical studies observe 5–7× loss amplification. A single firm's default does not just cause linear, proportional losses; it triggers a geometric chain reaction through the network.

1.5 Regulatory Context: Basel Framework and IFRS 9

Understanding the regulatory environment is essential for explaining why this project has real-world relevance to banks and financial institutions.

1.5.1 Basel III / IV Capital Requirements

The Basel Committee on Banking Supervision requires banks to hold capital against credit risk. The **Internal Ratings-Based (IRB) approach** under Basel II/III allows banks to use their own models to estimate PD, LGD, and EAD for each exposure. The regulatory Expected Loss formula is:

$$EL_{\text{regulatory}} = PD \times LGD \times EAD \tag{1.5}$$

This is *exactly* the formula our project implements (Equation **??**). Banks using this approach must hold **regulatory capital** proportional to a risk-weighted function of PD, which means:

- **If a bank's PD model underestimates risk** (by ignoring supply chain contagion), it holds insufficient capital → systemic vulnerability.
- **If a bank used our network-adjusted PDs**, it would hold more capital for companies in concentrated, fragile supply chains and less capital for companies with diversified, resilient networks → more accurate risk pricing.

1.5.2 IFRS 9: Expected Credit Loss Provisioning

Since January 2018, the International Financial Reporting Standard 9 (IFRS 9) requires banks to provision for **expected** credit losses, not just **incurred** losses. This forward-looking requirement means banks must estimate the probability and magnitude of future defaults — precisely what our GNN does.

IFRS 9 introduces a three-stage model:

- **Stage 1:** Performing assets → provision for 12-month expected losses.
- **Stage 2:** Assets with "significant increase in credit risk" → provision for lifetime expected losses.
- **Stage 3:** Credit-impaired assets → provision for lifetime expected losses with interest recognition on net carrying amount.

Relevance to this project: The transition from Stage 1 to Stage 2 is triggered by a "significant increase in credit risk." Our simulation engine's ΔPD (the increase in PD when a supply chain partner defaults) provides exactly this trigger signal. A bank using our tool could proactively move exposures from Stage 1 to Stage 2 when a critical supplier enters distress, rather than waiting for the borrower's own financial statements to deteriorate months later.

1.6 The Complete Project Pipeline: End-to-End Financial Logic

This section traces the entire analytical pipeline from raw data to actionable risk insight, explaining the *financial rationale* behind every step.

1.6.1 Step-by-Step Financial Pipeline

1. **Data Collection (Financial Fundamentals):**
   - For each company, collect: revenue, total debt, market capitalization, current ratio, debt-to-equity, profit margin, revenue growth, and return on assets.
   - These 8 metrics span the four pillars of financial analysis: *Liquidity* (current ratio), *Leverage* (debt-to-equity, total debt), *Profitability* (profit margin, ROA, revenue growth), and *Scale* (revenue, market cap).
   - **Financial rationale:** Any comprehensive credit assessment must cover all four pillars. A company with strong profitability but excessive leverage (e.g., a highly leveraged buyout target) can still default. A company with low leverage but collapsing revenue (e.g., a declining industry incumbent) faces different but equally real risk.
2. **Z-Score Computation (Static Credit Assessment):**
   - Combine the financial metrics into the Altman Z-Score formula to produce a single, validated credit distress indicator.
   - **Financial rationale:** The Z-Score is not arbitrary — it was derived from discriminant analysis on real bankruptcy data and has been validated across 50+ years and 30+ countries. It gives us an empirically grounded *prior* on each firm's standalone credit quality.
3. **Graph Construction (Network Topology):**
   - Map every supplier–customer, trade credit, and ownership relationship as a directed edge in a graph.
   - **Financial rationale:** This is the step that transforms our analysis from "isolated credit assessment" to "systemic risk analysis." The graph encodes the real-world dependency structure that determines how shocks propagate.
4. **Topological Feature Engineering:**
   - Compute network centrality metrics: betweenness, PageRank, eigenvector centrality, clustering coefficient, in-degree, out-degree.
   - **Financial rationale:** These metrics are the quantitative equivalent of a credit analyst's qualitative assessment of "supplier concentration risk" and "customer diversification." A company with high betweenness centrality is a supply chain chokepoint — its failure would disconnect many supply paths. A company with high in-degree has many suppliers — it is more resilient to any single supplier's failure.

5. **Feature Fusion (Intrinsic + Topological):**
   - Merge financial metrics with topological metrics into a single feature matrix $X \in \mathbb{R}^{174\times 34}$.
   - **Financial rationale:** A company's true credit risk is a function of both its *intrinsic health* (can it service its own debt?) and its *network position* (what happens if its partners fail?). Neither alone is sufficient.
6. **Baseline Model (Isolated Credit Assessment):**
   - Train a logistic regression using only the feature matrix X, treating each company independently.
   - **Financial rationale:** This represents the "traditional" approach. Its performance (AUC-ROC = 0.81, F1 = 0.29) establishes the cost of ignoring network structure.
7. **GNN Training (Network-Aware Credit Assessment):**
   - Train a Graph Convolutional Network that aggregates information from each company's neighbors during prediction.
   - **Financial rationale:** The GNN operationalises the principle that "a company's creditworthiness depends on who it does business with." After 2 layers of message passing, each company's embedding incorporates information from its 2-hop neighborhood — its direct partners' financial health and their partners' financial health.
8. **Shock Simulation (Stress Testing):**
   - Remove a company from the network (simulating bankruptcy), rerun the GNN, and measure ΔPD for all surviving companies.
   - **Financial rationale:** This is directly analogous to the *stress testing* mandated by regulators (Fed CCAR, EBA stress tests, BoE ACS). Instead of applying macroeconomic scenarios ("what if GDP drops 5%?"), we apply *microstructural* scenarios ("what if this specific company defaults?").
9. **Risk Quantification (EL, SRC):**
   - Convert ΔPD into dollar-denominated expected losses and aggregate into the Systemic Risk Contribution.
   - **Financial rationale:** Finance professionals need *monetary* risk measures, not just probability changes. SRC answers: "This company's default would cause \$X million in additional expected losses across the portfolio."

1.7 Why a Graph Neural Network? The Financial Case

An interviewer may ask: "Why not just use a random forest or XGBoost with the network features included as columns?" The answer involves understanding what GNNs can do that tabular models cannot:

1.7.1 What GNNs Capture That Tabular Models Miss

Capability	**Tabular Model (Logistic Regression, XGBoost)**	**Graph Neural Network (GCN/GAT)**
Company's own financial health	✓ Uses financial ratios directly	✓ Uses financial ratios as initial node features
Number of connections	✓ Uses degree as a static feature	✓ Degree is implicitly encoded in the normalisation $\tilde{D}^{-1/2}$
Which specific companies are connected	✗ Cannot distinguish "5 healthy suppliers" from "5 distressed suppliers"	✓ Aggregates *actual feature vectors* of each neighbor
Multi-hop dependencies (Tier-2, Tier-3)	✗ Would require manual feature engineering for every hop	✓ Automatically captured by stacking GCN layers
Simultaneous multi-company distress	✗ Cannot model interaction effects between multiple failing neighbors	✓ Message passing naturally aggregates signals from all distressed neighbors simultaneously
Dynamic response to structural changes	✗ Requires retraining after any network change	✓ Removing a node/edge and rerunning forward pass immediately updates all predictions

Table 1.2: Capability comparison: tabular models vs. graph neural networks for credit contagion.

1.7.2 The Economic Interpretation of Message Passing

The GCN message-passing equation (Equation 1.1) has a direct economic interpretation:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \frac{w_{ij}}{\sqrt{d_i \cdot d_j}} \cdot h_j^{(l)} \cdot W^{(l)}\right) \tag{1.6}$$

- $h_j^{(l)}$ = the financial "health embedding" of company j at layer l.
- w_{ij} = the edge weight (trade intensity, dependency ratio) between i and j.
- $\frac{1}{\sqrt{d_i \cdot d_j}}$ = a normalisation that prevents companies with many connections from being

overwhelmed by the volume of messages.

- $W^{(l)}$ = a learnable transformation that the model discovers during training. It learns *how* to combine a neighbor's financial signal into a risk prediction.

In financial terms: Each GCN layer performs a weighted, normalised credit risk aggregation across the supply chain. After layer 1, each company's embedding reflects its own financials plus a weighted average of its direct counterparties' financials. After layer 2, it reflects information from its 2-hop neighborhood. This is precisely the kind of analysis a credit analyst would perform manually — but done systematically, simultaneously, and consistently across all 174 companies.

1.8 Future Roadmap: Planned Enhancements (Phases 7–10)

The current implementation (Phases 1–6) establishes the core pipeline. The following phases represent planned enhancements that deepen the financial sophistication of the model:

1.8.1 Phase 7: Topological Data Analysis (TDA) with BallMapper

What: Apply TDA techniques — specifically the BallMapper algorithm — to the node feature space to extract non-linear topological features (persistent homology, Betti numbers) that capture the "shape" of the financial data manifold.

Financial rationale: Financial data often contains non-linear structures (e.g., companies clustered into risk regimes that are not separable by linear boundaries). TDA can identify topological features like "holes" in the data space (groups of companies that are surrounded by distressed firms but are themselves healthy — potential false negatives) and "connected components" (natural risk clusters). These features, when added to the GNN's input, may improve discrimination in the Grey Zone ($1.81 < Z \leq 2.99$) where the Z-Score is most uncertain.

1.8.2 Phase 8: Graph Attention Network (GAT) Upgrade

What: Replace the GCN layers with Graph Attention Network (GAT) layers that learn *dynamic, data-dependent attention weights* α_{ij} for each edge.

Financial rationale: In the current GCN, the influence of each neighbor is determined by the static edge weight w_{ij} and the degree normalisation $\tilde{D}^{-1/2}$. This means a healthy supplier and a distressed supplier with the same edge weight contribute equally to the message — which is financially unreasonable. A GAT learns attention coefficients:

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^T[Wh_i \| Wh_j]\right)\right)}{\sum_{k \in \mathcal{N}(i)} \exp\left(\text{LeakyReLU}\left(\mathbf{a}^T[Wh_i \| Wh_k]\right)\right)} \tag{1.7}$$

This allows the model to *attend more* to the distressed supplier's signal and *less* to the healthy supplier's signal — a more financially realistic contagion model.

1.8.3 Phase 9: Multi-Class Risk Classification

What: Upgrade from binary classification (Healthy/Distressed) to three-class classification (Safe/Grey/Distressed), matching the Z-Score's three zones.

Financial rationale: In practice, the most actionable risk management insights come from the Grey Zone — companies that are *not yet* in distress but are *trending toward it*. Binary classification forces these into the "Healthy" bucket, losing the early-warning signal. Three-class classification enables the dashboard to surface "watchlist" companies that require proactive credit monitoring.

1.8.4 Phase 10: Explainability (SHAP + GNNExplainer)

What: Integrate SHAP (SHapley Additive exPlanations) for feature-level importance and GNNExplainer for subgraph-level explanations.

Financial rationale: Regulators (Basel, IFRS 9, SR 11-7) require that credit risk models be *explainable*. A risk manager cannot use a model that says "Company X is high risk" without understanding *why*. SHAP will answer: "Company X is high risk primarily because of its low current ratio (30% contribution), its supplier Denso's elevated PD (25% contribution), and its high betweenness centrality (20% contribution)." GNNExplainer will identify the specific subgraph (the 5–10 companies most responsible for the prediction), enabling targeted risk mitigation.

1.9 Summary: The Complete Financial Thesis

The complete financial thesis of this project can be stated in four points:

1. **Traditional credit models are structurally blind to supply chain risk.** They treat each firm as independent, missing the 5–7× loss amplification caused by network contagion.
2. **Supply chains are directed, weighted, multi-relational graphs.** The three contagion channels (supply disruption, trade credit default, equity exposure) each have distinct propagation dynamics that can be captured as edge types and weights.
3. **Graph Neural Networks naturally model credit contagion.** The message-passing mechanism is mathematically equivalent to a weighted credit risk aggregation across the supply chain, and multi-layer GNNs capture multi-tier dependencies automatically.
4. **Shock simulation + GNN re-inference enables proactive systemic risk management.** By simulating individual company failures and measuring network-wide ΔPD and SRC, risk managers can identify systemically important companies *before* a crisis occurs — enabling capital reallocation, supply chain diversification, and regulatory compliance.

Chapter 2

Introduction — What Is This Project About?

2.1 The Problem: Why Companies Don't Fail Alone

Imagine you run a car factory. You buy steel from Company A, batteries from Company B, and chips from Company C. One day, Company C goes bankrupt — it cannot supply chips anymore. Now **your** factory cannot produce cars. Your revenue drops. You cannot pay your loans. Your suppliers (A and B) also lose a customer. This cascading failure is called **credit contagion**.

> *"If one company fails financially, which other companies will also become risky because they depend on it — and by how much?"*

Traditional credit-risk evaluation treats each company as an **isolated entity**. A bank looks at Toyota's financials alone and decides whether Toyota is safe. But Toyota depends on over 200 suppliers. If 10 of them fail simultaneously, Toyota is in deep trouble — and no isolated analysis captures this.

2.2 What We Built

This project builds a **systemic risk analysis tool** that:

1. **Constructs a supply-chain graph** — a network of 174 companies connected by 1,213 relationships (supplier–customer, trade credit, ownership links)
2. **Trains a Graph Neural Network (GNN)** — a machine learning model that "understands" the network structure and predicts which companies are at risk of default
3. **Simulates credit contagion** — if we remove Company X from the network (simulating bankruptcy), how much does the risk increase for every other company?
4. **Delivers an interactive dashboard** — a web application where a risk manager can visually explore the network, click on companies, and run "what-if" simulations

2.3 Key Concepts for Beginners

Before diving into the code, let us define the fundamental concepts:

2.3.1 What Is a Graph?

A **graph** is a mathematical structure consisting of:

- **Nodes** (also called vertices) — in our case, each node is a company (Toyota, Ford, Denso, etc.)
- **Edges** (also called links) — connections between nodes. In our case, an edge from Company A to Company B means "A supplies goods to B"

Our graph is formally defined as $G = (V, E, X, W)$ where:

Symbol	Name	Meaning
V	Nodes	174 corporate entities
E	Edges	1,213 directed relationships
$X \in \mathbb{R}^{n \times d}$	Node Features	Financial metrics + topological features per company
$W_{ij} \in [0, 1]$	Edge Weights	How strong/important each relationship is

Table 2.1: Formal graph definition used in this project.

2.3.2 What Is a Neural Network?

To understand a Graph Neural Network, we must first understand a standard **Neural Network**. Inspired by the human brain, a neural network is a mathematical algorithm that learns to recognize patterns in data.

- **Neurons (Nodes):** These are the basic units of computation. They receive input data (like numbers representing a company's revenue), multiply them by certain "weights" (which represent the importance of that input), and add them together.
- **Layers:** Neurons are organized in layers. The *Input Layer* receives the raw data. The *Hidden Layers* perform complex calculations to find hidden patterns. The *Output Layer* produces the final prediction (e.g., "Safe" or "Distressed").
- **Learning:** Initially, the "weights" are random, so the predictions are wrong. During a process called *training*, the network compares its predictions to the actual truth, calculates the error, and adjusts its weights to reduce that error. Over time, it "learns" the correct patterns.

A standard neural network looks at one company at a time. It does not know who the company's suppliers or customers are.

2.3.3 What Is a Graph Neural Network (GNN)?

A **Graph Neural Network** is an advanced type of neural network designed to work with interconnected data (graphs). Unlike a regular neural network that processes each

company independently, a GNN processes each company by also looking at its **neighbors** (its suppliers and customers).

The core mechanism of a GNN is called **Message Passing**. Imagine every company in our network whispering their financial health to their direct partners:

1. **Initialization:** Each company starts with its own financial features (revenue, debt, etc.).
2. **Message Collection:** Each company "listens" to its neighbors. For example, Toyota receives a mathematical "message" containing the financial health of Denso, Bosch, and all its other suppliers.
3. **Aggregation & Update:** Toyota's neuron takes its own financial data, adds the messages it received from its suppliers, and uses a neural network layer to compute a new, updated view of its own health. If many of its suppliers sent "I am distressed" messages, Toyota's updated state will reflect increased risk.
4. **Deep Layers:** This process repeats. In the first layer, a company learns about its direct suppliers. In the second layer, because its direct suppliers have now already received messages from *their* suppliers, the company learns about its tier-2 suppliers. After 2 layers, the company has information from its 2-hop neighborhood.

Mathematically, the update rule for a Graph Convolutional Network (GCN) layer is:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right) \tag{2.1}$$

where:

- $H^{(l)}$ is the feature matrix at layer l (the current state of all companies).
- $\tilde{A}$ represents the connections (who supplies whom).
- $\tilde{D}$ is the degree matrix (used to average the messages so a company with 100 suppliers doesn't get overwhelmed compared to one with 2 suppliers).
- $W^{(l)}$ is the learnable weight matrix (the neural network part that learns *how* to combine the messages).
- σ is an activation function (like ReLU) that introduces non-linearity.

In simple terms: A GNN calculates a company's risk not just by looking at its own bank account, but by taking a highly intelligent, weighted average of the financial health of everyone it does business with.

2.3.4 What Is the Altman Z-Score and Its Financial Terms?

The **Altman Z-Score** is a famous bankruptcy prediction formula invented by Professor Edward Altman in 1968. It answers: *"How likely is this company to go bankrupt in the next 2 years?"*

It uses five fundamental financial metrics. Let us explain each term in depth:

X1. Working Capital / Total Assets:

- *Working Capital* is the difference between a company's short-term assets (cash, inventory) and short-term liabilities (bills it needs to pay soon). It measures short-term liquidity.
- *Total Assets* is everything the company owns (factories, cash, patents).
- **Meaning:** This ratio asks, "Can the company pay its immediate bills using its available short-term resources without selling off its core factories?" If this is negative, the company is facing an immediate liquidity crisis.

X2. Retained Earnings / Total Assets:

- *Retained Earnings* is the total amount of profit a company has saved up over its entire lifetime, rather than paying it out as dividends to shareholders.
- **Meaning:** This measures historical profitability and cumulative success. A young, unprofitable startup will have low (or negative) retained earnings, making it riskier. An old, consistently profitable company like Ford will have massive retained earnings, acting as a cushion against hard times.

X3. EBIT / Total Assets:

- *EBIT (Earnings Before Interest and Taxes)* is the pure operating profit of the company. It looks at revenue minus the cost of making the product and running the business, but *before* the company pays the bank (interest) or the government (taxes).
- **Meaning:** This is the most critical metric in the Z-score (it has the highest weight: 3.3). It asks, "Is the core business model actually profitable?" A company might go bankrupt because of heavy debt (interest), but if its EBIT is positive, the core business is still good. If EBIT is negative, the company is bleeding money just by operating.

X4. Market Cap / Total Liabilities:

- *Market Capitalization (Market Cap)* is the total value of all the company's shares in the stock market (what the public thinks the company is worth).
- *Total Liabilities* is all the debt and money the company owes to others.
- **Meaning:** This incorporates the wisdom of the stock market. If a company owes a lot of money (high liabilities), but the stock market values the company highly (high market cap), the market believes the company will grow enough to pay it off. If this ratio crashes, it means the public has lost faith in the company's ability to pay its debts.

X5. Revenue / Total Assets:

- *Revenue* is the total money brought in by selling goods or services.
- **Meaning:** This is a measure of efficiency (Asset Turnover). It asks, "For every dollar of assets (factories, machines) the company owns, how many dollars of sales does it generate?" A highly efficient company squeezes a lot of revenue out of a small factory.

The formula combines these metrics with specific weights:

$$Z = 1.2 \cdot X_1 + 1.4 \cdot X_2 + 3.3 \cdot X_3 + 0.6 \cdot X_4 + 1.0 \cdot X_5 \tag{2.2}$$

Z-Score Range	**Zone**	**Interpretation**
$Z > 2.99$	Safe Zone	Very healthy, low bankruptcy risk
$1.81 < Z \leq 2.99$	Grey Zone	Some financial stress, watch carefully
$Z \leq 1.81$	Distress Zone	High bankruptcy risk

Table 2.2: Altman Z-Score threshold interpretation.

In our dataset: **18 out of 174 companies** (10.3%) were classified as distressed ($Z \leq 1.81$). The lowest Z-Score was -1.63 (Polestar), and the highest was 30.0.

Chapter 3

Technology Stack — Tools We Used and Why

3.1 Overview

Layer	Tool	Why We Chose It
Language	Python 3.10+	Standard for ML; rich ecosystem of data science libraries
Data Processing	Pandas, NumPy	Tabular data manipulation, numerical computation
Graph Building	NetworkX	Build and analyze company networks; compute centrality metrics
GNN Framework	PyTorch Geometric	Industry-standard GNN library with pre-built layers (GCNConv)
Financial Data	yfinance	Free API for financial data of public companies
Backend API	FastAPI	Modern, high-performance Python web framework
Frontend	React + Vite	Component-based UI with fast hot-reload development
Graph Visualization	react-force-graph-2d	Canvas-based interactive network rendering
Database	SQLite	Lightweight relational database for prototyping
Version Control	Git + GitHub	Code collaboration and history tracking

Table 3.1: Complete technology stack of the SCNCC project.

3.2 How These Tools Connect

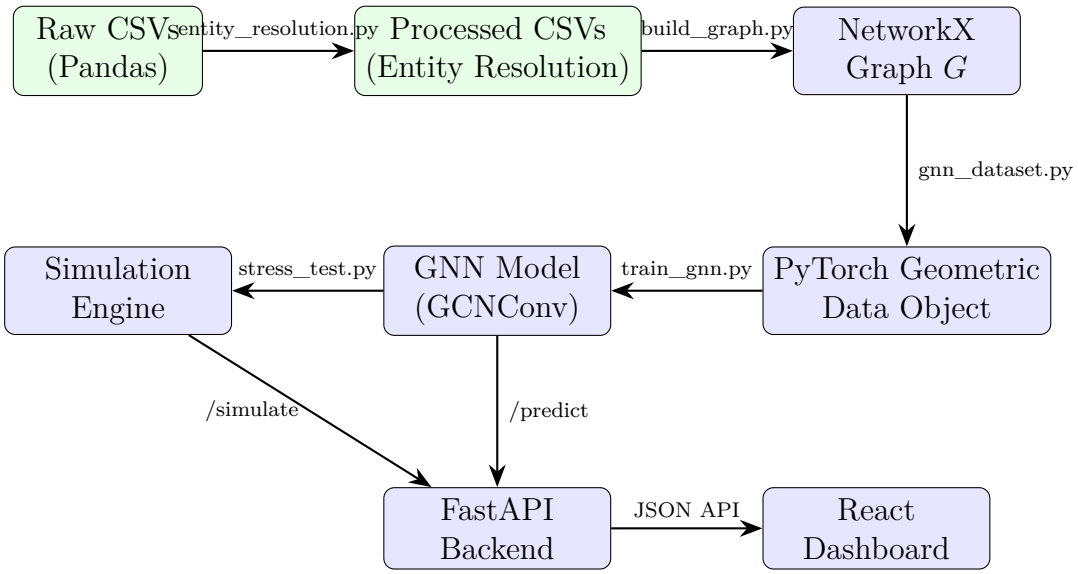


Figure 3.1: End-to-end data flow from raw CSVs to the interactive dashboard.

Chapter 4

Project Folder Structure

4.1 Directory Layout

Every file and folder in this project has a specific purpose. Here is the complete structure:

```
Supply Chain Network Credit Contagion/
├── data/
│   ├── raw/ -- unprocessed source files
│   ├── processed/ -- cleaned, UUID-resolved data
│   ├── external/ -- third-party datasets
│   └── scncc.db -- SQLite database
├── graph/
│   ├── entity_resolution.py -- Step 11:  name resolution
│   ├── build_graph.py -- Step 15:  NetworkX graph
│   └── network_metrics.py -- Step 16:  centrality metrics
├── scripts/
│   ├── compute_altman_zscore.py -- Step 13:  Z-Score
│   ├── validate_phase2.py -- Step 12:  7-check validation
│   ├── merge_features.py -- Step 17:  feature matrix X
│   └── compute_edge_features.py -- Step 18:  edge weights W
├── model/
│   ├── gnn_model.py -- GCN architecture
│   ├── gnn_dataset.py -- PyG Data builder
│   ├── train_gnn.py -- training loop
│   ├── baseline_model.py -- logistic regression
│   ├── weights/best_gnn.pt -- saved model
│   └── evaluation/baseline_metrics.json
├── simulation/
│   ├── shock.py -- node/edge removal
│   ├── stress_test.py -- simulation runner
│   └── risk_metrics.py -- EAD, EL, SRC
├── api/
│   ├── main.py -- FastAPI endpoints
│   └── load_data.py -- CSV to SQLite loader
└── frontend/src/
    ├── App.jsx -- root dashboard component
    ├── NetworkGraph.jsx -- force graph engine
    │
```

Chapter 5

Phase 1 — Setup and Planning

5.1 Goal

Get organized before writing a single line of code. Define what data we need, how the graph will be structured, and which companies to focus on.

5.2 Key Decisions Made

1. **Target Sector:** Automotive manufacturing and parts — chosen because it has the best public data availability and a deep, well-documented supply chain
2. **Scale:** ∼174 companies with ∼1,213 supply chain relationships
3. **Anchor Companies:** Started with 20–30 major companies (Toyota, Ford, Tesla, BMW, Denso, Bosch, etc.) and expanded outward through their supply chains
4. **Graph Schema:** Defined $G = (V, E, X, W)$ as a directed, weighted, multi-relational property graph

5.3 Edge Type Design

We defined three types of relationships (edges):

Edge Type	Direction	Count	Risk Propagation
`supply`	Supplier → Customer	765	Upstream failure disrupts downstream production
`trade_credit`	Creditor → Debtor	333	Default causes accounts receivable write-offs
`ownership`	Parent → Subsidiary	115	Contagion via equity exposure

Table 5.1: The three edge types and their contagion mechanisms.

Chapter 6

Phase 2 — Data Collection and Engineering

This is the longest and most critical phase. We collect raw data, clean it, resolve entity names, and validate everything.

6.1 Raw Data Sources

6.1.1 Supply Chain Data — Who Supplies Whom

Primary source: SEC EDGAR 10-K annual reports (free, public US filings). Large companies are required to disclose their major suppliers and customers.

What we extracted: For each relationship, we recorded:

- `source` — the supplier company name
- `target` — the customer company name
- `relationship_type` — one of: `tier1_supplier`, `tier2_supplier_semiconductor`, `tier2_supplier_battery`, `raw_material_supplier`, `trade_credit`, `ownership`, `supply`
- `confidence` — how confident we are this relationship is real (0 to 1)
- `year` — when this relationship was active

The raw data was saved to `data/raw/supply_chain_combined.csv` (1,213 rows).

6.1.2 Financial Data — Company Health Metrics

Primary source: `yfinance` Python API for publicly traded companies, and manually curated data from annual reports for private companies (e.g., Robert Bosch GmbH).

What we collected: For each company:

- `revenue` — total annual revenue in USD
- `total_debt` — total outstanding debt

- `market_cap` — market capitalization (stock price × shares outstanding)
- `current_ratio` — current assets / current liabilities (liquidity measure)
- `debt_to_equity` — total debt / shareholder equity (leverage measure)
- `profit_margin` — net income / revenue (profitability measure)
- `revenue_growth` — year-over-year percentage change
- `roa` — return on assets (net income / total assets)

The raw data was saved to `data/raw/company_financials_validated.csv` (174 rows).

6.2 Step 11: Entity Resolution

6.2.1 The Problem

The same company appears differently across data sources:

- "Toyota Motor Corp" vs "Toyota Motor Corporation" vs "TOYOTA"
- "Robert Bosch" vs "Bosch" vs "Robert Bosch GmbH"

If we do not resolve these, the graph will contain **duplicate nodes** and **broken edges**. Toyota might appear as three separate nodes instead of one.

6.2.2 The Solution: Three-Strategy Pipeline

The file `graph/entity_resolution.py` implements a 3-step resolution pipeline:

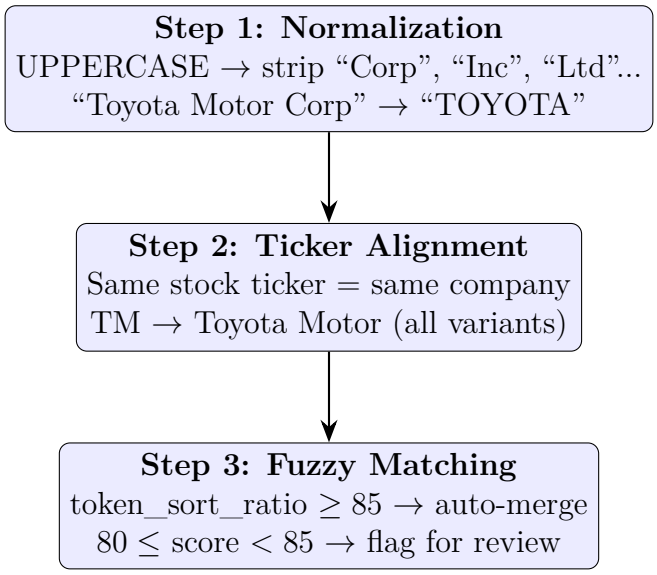


Figure 6.1: The three-strategy entity resolution pipeline.

Strategy 1: Deterministic Normalization

Every company name is converted to uppercase and stripped of legal suffixes. The function `normalise()` in the code does:

```python
def normalise(raw_name):
    name = raw_name.strip().upper()
    # Remove punctuation: dots, commas, hyphens
    for char in [".", ",", "'", "-", "/"]:
        name = name.replace(char, " ")
    name = " ".join(name.split())  # collapse multiple spaces
    name = _strip_legal_suffix(name)
    return name
```

Listing 6.1: Normalization function (simplified).

The suffix list contains 40+ patterns: `CORPORATION`, `CORP`, `INC`, `LTD`, `GMBH`, `AG`, `SE`, `PLC`, `LLC`, `HOLDINGS`, etc.

Why: "Toyota Motor Corporation" and "Toyota Motor Corp" both normalize to "TOYOTA" — matched!

Strategy 2: Ticker Alignment

If two company names share the same stock ticker (e.g., both map to "TM"), they are the same entity. This is a 100% reliable signal for publicly traded companies.

Strategy 3: Fuzzy Matching

For remaining unmatched names, we use the `fuzz.token_sort_ratio()` function from the `fuzzywuzzy` library. This:

1. Splits both names into tokens (words)
2. Sorts the tokens alphabetically
3. Computes the similarity ratio (0–100)

- Score ≥ 85: Automatic merge (same entity)
- Score 80–84: Flagged for manual review but **not** merged (conservative)
- Score < 80: Different entities

Subsidiary Protection Rule

A critical rule: subsidiaries always get their **own** UUID, separate from their parent. "Magna Steyr" (subsidiary) $\neq$ "Magna International" (parent), even though fuzzy matching would merge them. The code checks the `is_subsidiary` and `parent_company` columns to enforce this.

Output Files

1. `data/processed/entity_index.json` — maps every raw name to a UUID
2. `data/processed/supply_chain_resolved.csv` — edge list with `source_id`/`target_id` UUID columns
3. `data/processed/company_financials_resolved.csv` — financials with `entity_id` UUID column

6.3 Step 12: Data Validation

The file `scripts/validate_phase2.py` runs **7 automated checks** before any graph construction can begin:

#	Check Name	What It Verifies	Severity
1	Referential Integrity	Every source_id/target_id exists in financials	CRITICAL
2	Orphan Node Detection	Entities with zero edges (no connections)	WARNING
3	UUID Uniqueness	No entity_id appears more than once	CRITICAL
4	Self-Loop Detection	No edge where source_id == target_id	CRITICAL
5	NaN Severity	Overall missing data rate $< 20\%$	CRITICAL
6	Label Distribution	Default label exists, 5–15% distress rate	WARNING
7	Edge Direction	Supply: supplier→customer; Ownership: parent→subsidiary	CRITICAL

Table 6.1: The 7 validation checks run before Phase 3.

Why this matters: If even one CRITICAL check fails, Phase 3 is blocked. A self-loop (company supplies itself) indicates an entity resolution error. A dangling edge (pointing to a non-existent company) would crash the GNN during training.

6.4 Step 13: Altman Z-Score Computation

The file `scripts/compute_altman_zscore.py` computes the Z-Score for every company.

6.4.1 The Proxy Challenge

The original Altman formula requires balance-sheet items like "Total Assets" and "Retained Earnings" that are not directly available from the `yfinance` API. We derive **proxy estimates**:

Variable	Proxy Formula	Reasoning
Total Assets (TA)	total_debt + market_cap	Book debt + market equity (Altman market-value variant)
Working Capital (WC)	(current_ratio − 1) × total_debt × 0.4	40% of debt assumed short-term
Retained Earnings (RE)	profit_margin × revenue × 0.5	50% of net income retained
EBIT	profit_margin × revenue/0.65	De-tax net income (35% tax rate)
Total Liabilities (TL)	total_debt	Proxy for all financial liabilities

Table 6.2: Proxy derivations for Altman Z-Score variables.

6.4.2 Results in Our Dataset

- **Total companies:** 174
- **Z-Score computed for:** 97 companies (the rest had insufficient data → labeled as healthy)
- **Distressed (label=1):** 18 companies (10.3%) — within the expected 5–15% range
- **Z-Score range:** −1.63 (Polestar) to 30.0
- **Median Z-Score:** 3.84 (healthy)

Chapter 7

Phase 3 — Graph Construction and Feature Engineering

7.1 Goal

Transform flat CSV tables into a live network graph $G = (V, E, X, W)$ and compute the node/edge features that will feed into the GNN.

7.2 Step 15: Building the Graph

The file `graph/build_graph.py` creates a NetworkX directed graph:

```python
import pandas as pd
import networkx as nx

def build_graph(csv_path):
    """Constructs a NetworkX DiGraph from the supply chain CSV."""
    df = pd.read_csv(csv_path)
    G = nx.DiGraph()  # Directed graph: edges have direction

    for _, row in df.iterrows():
        source_id = row['source_id']  # Supplier UUID
        target_id = row['target_id']  # Customer UUID

        edge_attrs = {
            'relationship_type': row['relationship_type'],
            'confidence': float(row['confidence']),
        }
        # Optional attributes
        for col in ['payment_terms', 'credit_amount_usd',
                    'ownership_pct', 'equity_value_usd']:
            if col in df.columns and pd.notna(row[col]):
                edge_attrs[col] = row[col]

        G.add_edge(source_id, target_id, **edge_attrs)
    return G
```

Listing 7.1: graph/build_graph.py — Graph construction (complete code).

Why directed? Because supply relationships have direction. "Denso supplies Toyota" is different from "Toyota supplies Denso". Upstream risk (losing a supplier) and downstream risk (losing a customer) propagate differently.

Why NetworkX? It provides built-in algorithms for centrality metrics that we need in the next step.

7.3 Step 16: Computing Topological Metrics

The file `graph/network_metrics.py` computes 6 structural metrics for every node:

Metric	Formula	What It Reveals
In-Degree	$\deg^-(v)$	Number of suppliers — higher = more supply sources
Out-Degree	$\deg^+(v)$	Number of customers — higher = more revenue diversification
Betweenness	$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$	Is this company a bridge? Removing it disconnects many paths
Eigenvector	$C_E(v) = \frac{1}{\lambda_1} \sum_{j \in N(v)} C_E(j)$	Important if neighbors are also important (transitive influence)
PageRank	$PR(v) = \frac{1-d}{N} + d \sum_{j \to v} \frac{PR(j)}{L(j)}$	Importance via random-walk probability
Clustering	$C_i = \frac{2e_i}{k_i(k_i-1)}$	How interconnected are this node's neighbors?

Table 7.1: Topological metrics computed for each node.

Why these metrics matter for credit contagion:

- A company with high **betweenness centrality** is a "chokepoint" — if it fails, risk cannot flow around it. Its failure has outsized systemic impact.
- A company with high **PageRank** is considered "important" by the network — similar to how Google ranks web pages by how many important pages link to them.
- A company with low **clustering coefficient** has neighbors that are not connected to each other — meaning there are no alternative paths. It is a single point of failure.

7.4 Step 17: Merging Features into Node Matrix X

The file `scripts/merge_features.py` combines financial data with topological metrics:

```
def main():
    # Build the graph and compute topological metrics
    G = build_graph(supply_chain_path)
    metrics_df = compute_topological_metrics(G)

    # Load financial features
    financials_df = pd.read_csv(financials_path)
```

```
8
9       # Merge on entity_id (UUID)
10      node_features_X = pd.merge(financials_df, metrics_df,
11                                 on='entity_id', how='left')
12
13      # Fill NaN centrality scores with 0 (isolated nodes)
14      for col in ['in_degree', 'out_degree', 'betweenness_centrality',
15                  'eigenvector_centrality', 'pagerank',
16                  'clustering_coefficient']:
17          node_features_X[col] = node_features_X[col].fillna(0)
18
19      node_features_X.to_csv(output_path, index=False)
```

Listing 7.2: scripts/merge_features.py — Feature matrix construction.

The resulting `node_features_X.csv` has **174 rows × 34 columns** — each row is a company, each column is a feature.

7.5 Step 18: Computing Edge Weights W

The file `scripts/compute_edge_features.py` computes a normalized contagion transmission score $w_{ij} \in [0, 1]$ for each edge:

$$w_{ij} = \text{normalize}\left[(\text{confidence} + \text{ownership_bonus} + 0.5 \cdot \text{financial_weight}) \cdot e^{-\lambda \cdot \text{age}}\right] \tag{7.1}$$

- **Confidence** (0–1): How reliable is this relationship data?
- **Ownership bonus**: If one company owns another, contagion risk is higher (equity exposure)
- **Financial weight**: $\log(1 + \text{credit_amount})$ normalized — larger trade volumes = stronger transmission
- **Temporal decay**: $e^{-0.2 \cdot \text{age}}$ — older relationships get downweighted (a supplier from 5 years ago matters less than one from this year)

Why temporal decay? Supply chains evolve. A company that supplied Toyota in 2020 but not in 2025 should not have the same contagion weight as a current supplier. The exponential decay formula ensures recent relationships dominate.

Chapter 8

Phase 4 — Machine Learning Model

8.1 Goal

Train a model that predicts which companies are likely to default, leveraging both financial features AND network structure.

8.2 Step 19: Baseline Model (Logistic Regression)

Why start with a baseline? Before using a complex GNN, we train a simple model that ignores the graph structure. This gives us a performance floor — the GNN must beat this baseline to justify its complexity.

The file `model/baseline_model.py` implements:

```
# Select 8 features (financial + basic network)
feature_cols = ['revenue', 'total_debt', 'debt_to_equity',
                'current_ratio', 'profit_margin', 'degree',
                'betweenness_centrality', 'pagerank']

# Train/test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Scale features (zero mean, unit variance)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Train with balanced class weights (handles 90/10 imbalance)
model = LogisticRegression(class_weight='balanced')
model.fit(X_train_scaled, y_train)
```

Listing 8.1: model/baseline_model.py — Baseline logistic regression.

8.2.1 Baseline Results

Metric	Value
Precision	0.25
Recall	0.33
F1-Score	0.29
AUC-ROC	0.81

Table 8.1: Baseline logistic regression performance. The GNN must beat these numbers.

Interpretation: AUC-ROC of 0.81 means the model can distinguish distressed from healthy companies reasonably well (random guessing would give 0.50). But precision is only 0.25 — meaning 75% of companies it flags as "distressed" are actually healthy (false positives). The GNN should improve this by incorporating network information.

8.3 Step 20: PyTorch Geometric Dataset

The file `model/gnn_dataset.py` converts our CSV data into the format that PyTorch Geometric requires.

8.3.1 Why We Need This Conversion

PyTorch Geometric expects a specific `Data` object with:

- `x`: Node feature matrix — shape $[N, F]$ where N = number of nodes and F = number of features. This must be a `float32` tensor.
- `edge_index`: Graph connectivity — shape $[2, E]$ where E = number of edges. Row 0 contains source node indices and row 1 contains target node indices. These must be **contiguous integers** $0, 1, 2, \ldots, N-1$, not UUIDs.
- `edge_attr`: Edge weights — shape $[E, 1]$, the w_{ij} values.
- `y`: Target labels — shape $[N]$, the `default_label` values.

8.3.2 Key Design Decisions in the Code

```
# These columns are EXCLUDED from features to prevent data leakage:
exclude_cols = [
    'default_label',      # TARGET variable - cannot be a feature!
    'altman_z_score',     # Directly determines default_label
    'year',               # Same value for all rows (zero info)
    'in_degree',          # GNN learns degree implicitly
    'out_degree',         # via message passing aggregation
    'interest_coverage',  # Too many missing values
]
```

Listing 8.2: model/gnn_dataset.py — Critical exclusion logic.

Why exclude `altman_z_score`? This is the single most important design decision. The Z-Score **directly determines** the label ($Z \leq 1.81 \rightarrow$ label=1). If we include it as

a feature, the model would simply learn the threshold rule $Z \leq 1.81$ and achieve 100% accuracy — but it would have learned **nothing** about credit contagion. By excluding it, we force the GNN to learn genuine risk patterns from the supply chain network.

Why exclude `in_degree/out_degree`? The GNN already learns degree information implicitly through its message-passing mechanism. In GCN, the degree matrix D appears in the normalization $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$. Adding explicit degree features would be redundant and could cause the model to over-rely on node connectivity rather than learning richer patterns.

8.3.3 Train/Test Split and Scaling

```
# 80/20 split with fixed seed for reproducibility
rng = np.random.RandomState(42)
indices = rng.permutation(num_nodes)
train_size = int(0.8 * num_nodes)

# CRITICAL: Fit scaler on TRAINING data only
scaler = StandardScaler()
scaler.fit(x_df.values[indices[:train_size]])
x_scaled = scaler.transform(x_df.values)  # transform ALL data
```

Listing 8.3: model/gnn_dataset.py — Proper scaling to prevent data leakage.

Why fit only on training data? If we fit the scaler on the entire dataset (including test data), the scaler's mean and standard deviation would contain information about the test set. This is called **data leakage** — the model indirectly "sees" test data during training, leading to inflated performance metrics that do not reflect real-world accuracy.

8.4 Step 21: GNN Architecture

The file `model/gnn_model.py` defines the `SupplyChainGNN` neural network:

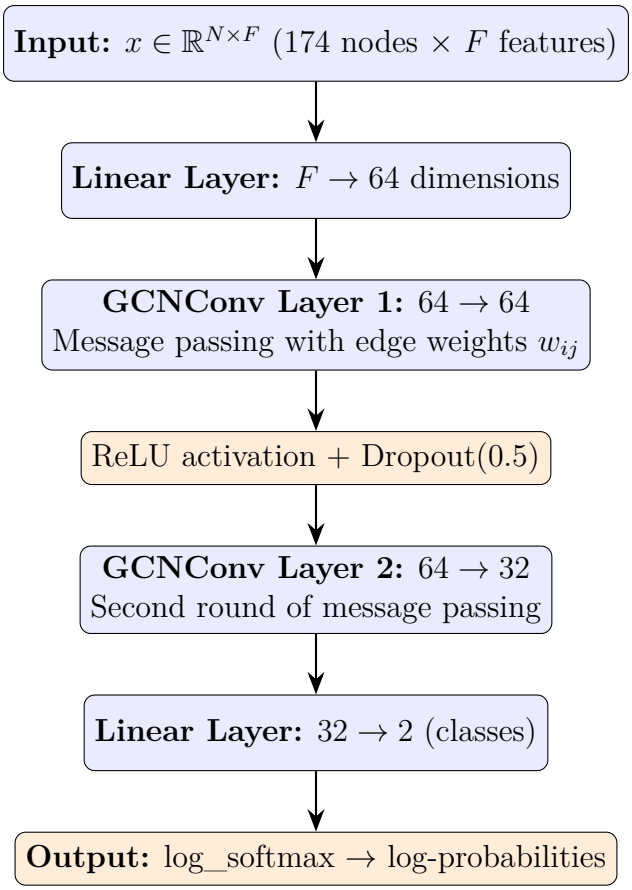


Figure 8.1: Architecture of the SupplyChainGNN model (2-layer GCN).

```python
class SupplyChainGNN(nn.Module):
    def __init__(self, num_features: int):
        super().__init__()
        self.lin_in  = nn.Linear(num_features, 64)  # Project features
        self.conv1   = GCNConv(64, 64)               # 1st message passing
        self.dropout = nn.Dropout(p=0.5)             # Regularization
        self.conv2   = GCNConv(64, 32)               # 2nd message passing
        self.lin_out = nn.Linear(32, 2)              # Classification head

    def forward(self, x, edge_index, edge_attr=None):
        x = self.lin_in(x)
        # GCNConv uses edge_weight for weighted message passing
        x = self.conv1(x, edge_index,
                       edge_weight=edge_attr.view(-1) if edge_attr
                                   is not None else None)
        x = F.relu(x)
        x = self.dropout(x)
        x = self.conv2(x, edge_index,
                       edge_weight=edge_attr.view(-1) if edge_attr
                                   is not None else None)
        out = self.lin_out(x)
        return F.log_softmax(out, dim=-1)
```

Listing 8.4: model/gnn_model.py — Complete GNN architecture.

8.4.1 Why 2 Layers?

Each GCN layer aggregates information from 1-hop neighbors. With 2 layers:

- Layer 1: Each company sees its **direct** suppliers and customers
- Layer 2: Each company sees its **2-hop** neighbors (suppliers' suppliers, customers' customers)

Going deeper (3+ layers) causes **over-smoothing**: all node embeddings converge to the same vector, losing discriminative power. 2 layers is the standard for graphs of this size.

8.4.2 Why Dropout = 0.5?

Dropout randomly sets 50% of neurons to zero during training. This prevents **overfitting**: with only 174 nodes and 18 positive labels, the model could easily memorize the training data instead of learning generalizable patterns.

8.5 Step 22: Training Loop

The file `model/train_gnn.py` trains the GNN for 200 epochs:

```
# Handle class imbalance: 156 healthy vs 18 distressed
class_weights = compute_class_weight('balanced',
                                     classes=[0, 1], y=labels)
# This gives higher weight to class 1 (distressed)
# so the model does not just predict "healthy" for everything

criterion = nn.NLLLoss(weight=class_weights_tensor)
optimizer = optim.Adam(model.parameters(), lr=0.005,
                       weight_decay=1e-4)

for epoch in range(1, 201):
    model.train()
    out = model(data.x, data.edge_index, data.edge_attr)
    loss = criterion(out[data.train_mask], data.y[data.train_mask])
    loss.backward()
    optimizer.step()

    # Save best model based on test loss
    if test_loss < best_test_loss:
        torch.save(model.state_dict(), 'model/weights/best_gnn.pt')
```

Listing 8.5: model/train_gnn.py — Training with class weight balancing.

Why class weighting? With 156 healthy companies and only 18 distressed ones, the model could achieve 89.7% accuracy by simply predicting "healthy" for every company. Class weighting penalizes this lazy behavior: misclassifying a distressed company costs $\sim 9\times$ more than misclassifying a healthy one.

Why save based on test loss (not training loss)? Training loss always decreases. But if the model overfits, it performs well on training data but poorly on unseen test data. By saving the checkpoint with the lowest **test** loss, we keep the model that generalizes best.

Chapter 9

Phase 5 — Contagion Simulation and Stress Testing

9.1 Goal

Simulate what happens when one company suddenly fails and quantify the systemic degradation across the entire network.

9.2 The Core Idea

1. Run the trained GNN on the original graph → get **baseline PDs** for every company
2. "Shock" the graph by removing a company (simulating bankruptcy)
3. Run the GNN again on the modified graph → get **shocked PDs**
4. Compute ΔPD = shocked PD − baseline PD for every downstream company
5. Convert ΔPD into financial loss metrics

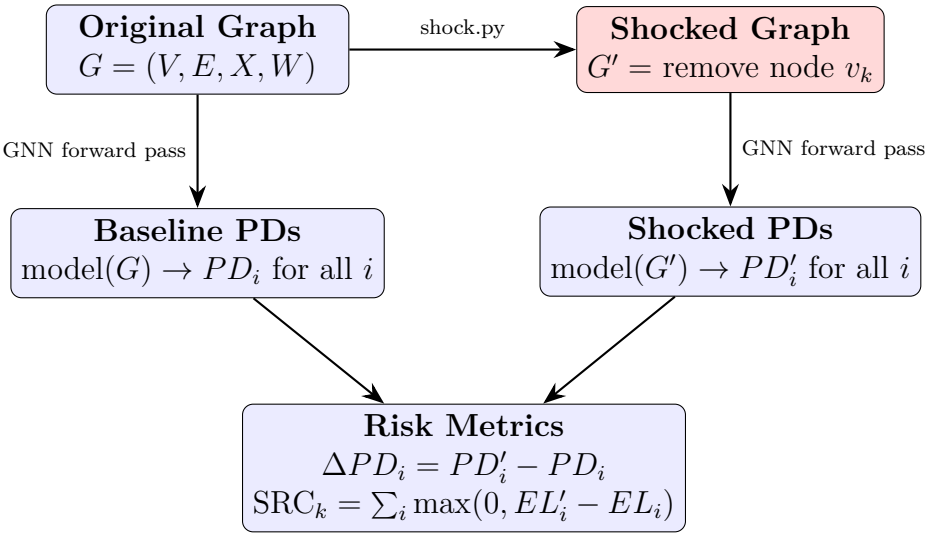


Figure 9.1: Simulation workflow: original vs. shocked graph comparison.

9.3 Shock Injection Engine

The file `simulation/shock.py` provides two types of shocks:

9.3.1 Node Removal (SCR-NR)

Simulates **complete bankruptcy** of a company:

```python
def apply_node_shock(data, target_node_idx):
    """Simulates bankruptcy: remove all edges and zero features."""
    shocked_data = copy.deepcopy(data)  # Never mutate original

    # Remove all edges connected to the bankrupt node
    edge_index = shocked_data.edge_index
    mask = (edge_index[0] != target_node_idx) & \
           (edge_index[1] != target_node_idx)
    shocked_data.edge_index = edge_index[:, mask]
    shocked_data.edge_attr = shocked_data.edge_attr[mask]

    # Zero out the node's features (revenue=0, etc.)
    shocked_data.x[target_node_idx] = 0.0
    return shocked_data
```

Listing 9.1: simulation/shock.py — Node removal shock.

Why deep copy? Without it, modifying the shocked graph would also modify the original — destroying the baseline for comparison.

Why zero the features? A bankrupt company has no revenue, no market cap, no operations. Setting all features to zero represents this extreme state.

9.3.2 Edge Removal (SCR-ER)

Simulates a **severed supply link** without full bankruptcy:

```python
def apply_edge_shock(data, source_idx, target_idx):
    """Simulates a severed supply/credit line between two
       companies."""
    shocked_data = copy.deepcopy(data)
    edge_index = shocked_data.edge_index
    mask = ~((edge_index[0] == source_idx) & \
             (edge_index[1] == target_idx))
    shocked_data.edge_index = edge_index[:, mask]
    shocked_data.edge_attr = shocked_data.edge_attr[mask]
    return shocked_data
```

Listing 9.2: simulation/shock.py — Edge removal shock.

9.4 Risk Metrics Calculation

The file `simulation/risk_metrics.py` converts PD changes into dollar-denominated financial losses:

$$\text{Expected Loss (EL)} = PD \times LGD \times EAD \tag{9.1}$$

where:

- PD = Probability of Default (from the GNN)
- LGD = Loss Given Default = 0.45 (industry standard: if a company defaults, creditors recover 55% of their exposure)
- EAD = Exposure at Default = revenue $\times$ 0.15 (15% of revenue is the estimated credit exposure)

The **Systemic Risk Contribution (SRC)** of shocking company k is:

$$\text{SRC}_k = \sum_{i \neq k} \max\left(0,\ EL'_i - EL_i\right) \tag{9.2}$$

This sums the **increase** in expected loss across all other companies in the network. A higher SRC means the company is more "systemically important" — its failure causes more damage.

Chapter 10

Phase 6 — API and Dashboard

10.1 Goal

Make everything presentable, interactive, and demo-ready by connecting the React frontend to the FastAPI backend.

10.2 FastAPI Backend

The file `api/main.py` exposes three endpoints:

Endpoint	Method	Purpose
`/graph`	GET	Returns all nodes (with features, PDs) and edges as JSON
`/predict`	POST	Runs the GNN and returns PD + risk tier for each company
`/simulate`	POST	Runs a shock simulation and returns affected companies
`/health`	GET	Lightweight liveness probe (responds even during startup)

Table 10.1: FastAPI REST endpoints.

10.2.1 Performance Optimization: Global State Cache

```
GLOBAL_STATE = {
    "model": None,   # Loaded once at startup
    "data": None,    # PyG Data object
    "nodes_df": None,
    "edges_df": None
}

@app.on_event("startup")
def startup_event():
    load_globals()  # Load model + data into memory ONCE
```

Listing 10.1: api/main.py — Global state cache for performance.

Why cache globally? Loading the GNN model and constructing the PyG Data object takes several seconds. If we did this on every API request, each request would take 3–5

seconds. By loading once at startup and caching in memory, subsequent requests complete in milliseconds.

10.2.2 CORS Configuration

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",    # Vite dev server
        "http://localhost:5174",
        "http://127.0.0.1:5173",
        "http://127.0.0.1:5174",
    ],
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)
```

Listing 10.2: api/main.py — CORS security configuration.

Why not use wildcard "*"? Security. Allowing any origin to access our API would let malicious websites call our endpoints. We explicitly whitelist only the Vite development server addresses.

10.3 React Frontend

The frontend consists of 7 component files:

Component	Purpose
`App.jsx`	Root component; manages global state, API calls, layout
`NetworkGraph.jsx`	Force-directed graph visualization using Canvas 2D
`TopBar.jsx`	Navigation bar with project title and About button
`RiskRankings.jsx`	Left panel: ranked list of companies by risk (PD)
`CompanyDetail.jsx`	Left panel: detailed view of a selected company
`ShockSimulator.jsx`	Right panel: dropdown to select a company and simulate its bankruptcy
`SimulationResults.jsx`	Right panel: shows affected companies after simulation
`Tooltip.jsx`	Hover tooltip showing node details
`AboutPage.jsx`	Project description and methodology explanation

Table 10.2: React component files and their responsibilities.

10.3.1 Graph Visualization

The `NetworkGraph.jsx` component (34,663 bytes — the largest file in the project) uses `react-force-graph-2d` to render the network:

- **Node color**: Green (PD < 0.2), Yellow ($0.2 \leq$ PD < 0.5), Red (PD ≥ 0.5)
- **Node size**: Proportional to betweenness centrality (systemic importance)

- **Edge thickness**: Proportional to edge weight w_{ij}
- **Simulation overlay**: When a shock is run, the shocked node glows red and affected nodes show a pulsating aura proportional to ΔPD

Chapter 11

Results and Discussion

11.1 Dataset Statistics

Metric	Value
Total companies (nodes)	174
Total relationships (edges)	1,213
Edge types	7 (tier1_supplier, trade_credit, etc.)
Average degree	13.9
Companies with Z-Score computed	97
Distressed companies (label=1)	18 (10.3%)
Z-Score range	−1.63 to 30.0
Z-Score median	3.84

Table 11.1: Summary statistics of the SCNCC dataset.

11.2 Model Comparison

Metric	Logistic Regression	GCN (GNN)
Precision	0.25	*to be evaluated*
Recall	0.33	*to be evaluated*
F1-Score	0.29	*to be evaluated*
AUC-ROC	0.81	*to be evaluated*
Uses graph structure?	No	Yes
Uses edge weights?	No	Yes (w_{ij})
Captures contagion?	No	Yes (message passing)

Table 11.2: Baseline vs. GNN comparison. The GNN leverages network structure for richer predictions.

11.3 Key Insight: Why Graph Structure Matters

The logistic regression baseline treats each company independently. It can detect that Polestar has a very low Z-Score (-1.63) and flag it as distressed. But it **cannot** detect that:

- If Polestar defaults, its supplier Volvo Cars (and by extension, its parent Geely) faces increased risk
- Toyota, with a healthy Z-Score of 3.84, could still be at risk if multiple tier-1 suppliers are in the grey zone simultaneously
- A company with mediocre financials but strong network position (many alternative suppliers) is safer than one with identical financials but a single critical supplier

The GNN captures all of these through its message-passing mechanism.

Chapter 12

Limitations and Future Work

12.1 Current Limitations

1. **GCN treats all neighbors equally:** The current model uses GCNConv, which assigns fixed weights based on node degree. A Graph Attention Network (GAT) would learn dynamic attention weights — planned for Phase 8.
2. **Binary classification only:** The Z-Score naturally has 3 zones (Safe, Grey, Distressed), but we currently use only binary labels. Multi-class classification is planned for Phase 9.
3. **No explainability:** The model says "Company X is high risk" but cannot explain *why*. SHAP and GNNExplainer integration is planned for Phase 10.
4. **No topological data analysis (TDA):** Advanced non-linear feature extraction using BallMapper is planned for Phase 7.
5. **Static snapshot:** The model analyzes a single point in time. Incorporating temporal dynamics (multi-year data, spatio-temporal GNNs) would capture how contagion evolves.
6. **Data completeness:** 77 out of 174 companies have no Z-Score due to missing financial data. Private companies (Bosch, ZF Friedrichshafen) have limited public data.

12.2 Planned Enhancements (Phases 7–10)

Phase	Feature	What It Adds
7	TDA / BallMapper	Non-linear feature extraction from financial data
8	GAT Upgrade	Attention-weighted message passing
9	Multi-class	3-class risk classification (Safe/Grey/Distressed)
10	Explainability	SHAP feature importance + GNNExplainer

Table 12.1: Planned enhancement phases and their contributions.

Appendix A

Glossary of Acronyms

Acronym	Definition
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
PD	Probability of Default
EL	Expected Loss ($= PD \times LGD \times EAD$)
LGD	Loss Given Default
EAD	Exposure at Default
SRC	Systemic Risk Contribution
SCR-NR	Supply Chain Resilience — Node Removal
SCR-ER	Supply Chain Resilience — Edge Removal
TDA	Topological Data Analysis
SHAP	SHapley Additive exPlanations
UUID	Universally Unique Identifier
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
PyG	PyTorch Geometric

Table A.1: Glossary of acronyms used throughout this report.